

Angular Moderno

20 - Regressao e Reorganizacao

2026

SENAI - CRTI

20 - Correção de Regressão e Reorganização da Comunicação entre Componentes





Contexto do Projeto

Durante a evolução do projeto de e-commerce, adicionamos diversas funcionalidades modernas que impactaram partes já existentes do sistema.

HTTP

Integração com APIs externas via `HttpClient` moderno

Service

Camada de serviço para separação de responsabilidades

Interceptor

Interceptação de requisições com providers funcionais

Tratamento de Erros

Feedback visual de erros diretamente na UI

⚠ Essas alterações impactaram partes já existentes do sistema, quebrando funcionalidades que antes funcionavam corretamente.

Problema Identificado

Após as refatorações realizadas nas aulas anteriores, dois problemas críticos foram introduzidos no projeto:

Botão "Selecionar Produto"

O evento não estava sendo capturado pelo componente pai.
Nenhuma ação ocorre ao clicar.

Botão "Adicionar ao Carrinho"

O botão desapareceu da interface. O código foi removido durante as refatorações.

Comunicação Quebrada

A comunicação entre componentes pai e filho foi interrompida.

Conceito Importante: Regressão

Refatoração pode introduzir regressões — e isso é mais comum do que parece no desenvolvimento de software real.

O que é uma Regressão?

Uma **regressão** ocorre quando uma funcionalidade que já funcionava corretamente **deixa de funcionar após alterações no código**, mesmo que essas alterações não tenham sido feitas diretamente nessa funcionalidade.

Por que acontece?

Ao refatorar ou adicionar novas features, é possível quebrar conexões existentes entre componentes, remover código necessário sem perceber, ou alterar contratos de comunicação sem atualizar todos os pontos de uso.

Como evitar?

Testes automatizados, revisão de código cuidadosa e verificação manual das funcionalidades existentes após cada alteração são as principais estratégias de prevenção.

Diagnóstico Técnico

lista-produtos.html

Ao analisar o template do componente pai, identificamos o problema raiz da regressão:

Código Atual (com problema)

```
<app-produto  
  [nome]="item.nome"  
  [preco]="item.preco"  
>
```

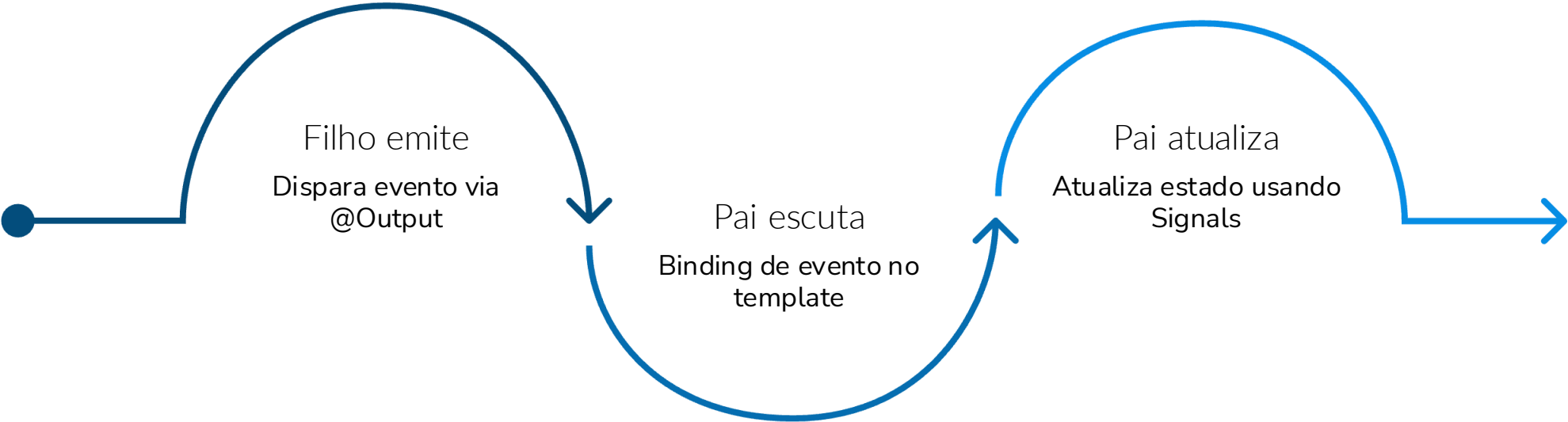
O que está faltando?

- O componente filho `produto` emite eventos via `@Output()`
- O componente pai **NÃO está escutando** nenhum desses eventos
- Sem o binding de evento (`produtoSelecionado`), nenhuma ação ocorre
- O fluxo de comunicação está completamente interrompido
- Esse foi o ponto exato da quebra no projeto

❌ Apenas passar dados via `[Input]` não é suficiente. É necessário também capturar os eventos emitidos pelo filho com `(evento)="metodo($event)"`.

Conceito: Comunicação entre Componentes

No Angular moderno, a comunicação entre componentes pai e filho segue um fluxo bem definido que deve ser respeitado para que o sistema funcione corretamente.



Sem o binding de evento no template do pai, o ciclo é interrompido na primeira etapa — o filho emite, mas ninguém escuta.



@Output no Filho

O componente `produto` declara um `@Output()` e emite eventos quando o usuário interage.



Binding no Pai

O componente `lista-produtos` usa `(evento)="metodo($event)"` para capturar o que o filho emitiu.



Signal Atualizado

O método no pai chama `.set()` ou `.update()` no Signal, e a UI reage automaticamente.

Etapa 1: Corrigir Seleção de Produto

lista-produtos.html

A primeira correção é adicionar o binding do evento `produtoSelecionado` no template do componente pai:

Antes (quebrado)

```
<app-produto  
  [nome]="item.nome"  
  [preco]="item.preco"  
>
```

Depois (corrigido)

```
<app-produto  
  [nome]="item.nome"  
  [preco]="item.preco"  
  (produtoSelecionado)=  
    "exibirProduto($event)"  
>
```

- ✅ Agora o evento emitido pelo componente filho é capturado pelo componente pai. O método `exibirProduto($event)` será chamado sempre que o usuário clicar em "Selecionar produto".

Explicação da Correção

lista-produtos.ts

O método `exibirProduto` já existia no componente pai. Ele apenas precisava ser reconectado ao evento do filho:

```
exibirProduto(nome: string) {  
  this.produtoSelecionado.set(nome);  
}
```

01

Evento carrega o dado

O evento `produtoSelecionado` emitido pelo filho carrega o **nome do produto** como payload via `$event`.

02

Signal é atualizado

O método chama `this.produtoSelecionado.set(nome)`, atualizando o valor do Signal com o nome recebido.

03

UI reage automaticamente

Como o template usa o Signal diretamente, o Angular detecta a mudança e atualiza a interface sem necessidade de `ChangeDetection` manual.

Novo Problema Identificado

Mesmo após corrigir a seleção de produto, identificamos um segundo problema:

Botão Desapareceu

O botão "**Adicionar ao carrinho**" não existe mais na interface. Ele foi removido durante as refatorações anteriores.

Causa Raiz

O código do botão e do `@Output()` correspondente foram removidos do componente `produto` durante a evolução do projeto.

Solução

Precisamos recriar o `@Output()`, o método de emissão e o botão no template — passo a passo.



Etapa 2: Criar Output para o Carrinho

produto.ts

O primeiro passo é declarar o `@Output()` no componente filho para que ele possa comunicar ao pai que um produto foi adicionado:

```
@Output() produtoAdicionado = new EventEmitter<{  
  nome: string;  
  preco: number;  

```

Por que **EventEmitter** tipado?

Ao tipar o **EventEmitter** com `{ nome: string; preco: number }`, garantimos que o componente pai receberá exatamente os dados necessários para adicionar o produto ao carrinho, com segurança de tipos em tempo de compilação.

Responsabilidade do filho

O componente **produto** não deve gerenciar o carrinho diretamente. Sua responsabilidade é apenas **informar ao pai** que o usuário deseja adicionar o produto — seguindo o princípio de responsabilidade única.

Etapa 3: Criar Método de Emissão

produto.ts

Com o `@Output()` declarado, criamos o método que será chamado quando o usuário clicar no botão.

```
adicionarAoCarrinho() {  
  this.produtoAdicionado.emit({  
    nome: this.nome,  
    preco: this.preco  
  });  
}
```

Por que encapsular em um método?

Encapsular a emissão em um método mantém o template limpo e o código organizado. Futuramente, podemos adicionar validações ou lógica extra sem alterar o template.

O que o método faz?

- Acessa `this.nome` e `this.preco` (Inputs do componente)
- Monta o objeto com os dados do produto
- Emite o evento via `.emit()`
- O pai receberá esse objeto via `$event`

Etapa 4: Adicionar Botão no Template

produto.html

Com o método criado, adicionamos o botão no template do componente filho para que o usuário possa interagir:
(Adicionar abaixo do botão 'Selecionar produto')

```
<button (click)="adicionarAoCarrinho()">  
  Adicionar ao carrinho  
</button>
```

Event Binding (**click**)

O (**click**) é um event binding do Angular que escuta o evento nativo de clique do DOM e chama o método especificado no componente.

Interface restaurada

Com o botão de volta no template, a interface volta a permitir a ação do usuário. O fluxo completo agora pode ser executado: clique → método → emissão → pai reage.

Separação de responsabilidades

O template apenas chama o método. Toda a lógica de emissão fica no TypeScript. Isso facilita testes e manutenção futura do componente.

Etapa 5: Capturar Evento no Componente Pai

lista-produtos.html

O passo final é atualizar o template do pai para escutar os **dois eventos** emitidos pelo filho:

```
<app-produto
  [nome]="item.nome"
  [preco]="item.preco"
  (produtoSelecioneado)="exibirProduto($event)"
  (produtoAdicionado)="adicionarAoCarrinho($event)"
/>
```

Dois eventos, dois bindings

Agora o componente pai reage a **ambos os eventos** do filho. Cada evento tem seu próprio método handler, mantendo o código organizado e com responsabilidades bem definidas.

Resumo dos bindings

- `[nome]` e `[preco]` → dados fluem do pai para o filho
- `(produtoSelecioneado)` → evento flui do filho para o pai
- `(produtoAdicionado)` → evento flui do filho para o pai

Método Já Existente no Pai

lista-produtos.ts

Uma boa notícia: o método `adicionarAoCarrinho` já existia no componente pai. Não precisamos recriar a lógica — apenas reconectar o fluxo: **Este método já existia no projeto e não foi alterado.**

```
adicionarAoCarrinho(produto: { nome: string; preco: number }) {  
  this.carrinho.update((listaAtual) => [...listaAtual, produto]);  
}
```

Signal `carrinho`

O carrinho é um Signal do tipo array. O método usa `.update()` para adicionar o novo produto sem mutar o array original.

Spread Operator

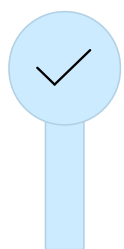
O `[...listaAtual, produto]` cria um novo array com todos os itens anteriores mais o novo produto — padrão imutável recomendado com Signals.

Sem duplicação de lógica

Não precisamos recriar nada. A lógica estava lá — apenas desconectada. Isso reforça a importância de entender o fluxo antes de deletar código.

Resultado da Correção

Após todas as alterações realizadas nas etapas anteriores, o sistema está completamente restaurado:



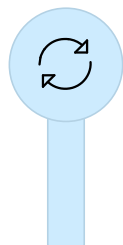
Seleção de produto funciona

O evento `produtoSelecionado` está conectado e o Signal é atualizado corretamente ao clicar.



Botão de adicionar reaparece

O botão "Adicionar ao carrinho" voltou ao template e está funcional para o usuário.



Carrinho volta a ser atualizado

O Signal `carrinho` é atualizado corretamente via `.update()` ao adicionar produtos.



Fluxo completo restaurado

A comunicação pai ↔ filho está funcionando nos dois sentidos: dados via `@Input` e eventos via `@Output`.



Aprendizado Importante

Sempre que refatorar, verifique se os eventos continuam conectados e teste as funcionalidades existentes.

Este é um checklist mental que todo desenvolvedor Angular deve seguir após qualquer refatoração:

1

Verifique os eventos

Confirme que todos os bindings de `@Output()` ainda estão presentes no template do componente pai após qualquer alteração.

2

Teste funcionalidades existentes

Antes de considerar uma refatoração concluída, teste manualmente (ou com testes automatizados) todas as funcionalidades que já existiam.

3

Valide o fluxo completo

Garanta que o fluxo **UI** → **evento** → **estado** → **UI** continua válido de ponta a ponta após cada mudança no código.

i No Angular moderno com Signals, o fluxo reativo depende que todos os elos da cadeia estejam conectados. Um binding faltando quebra toda a reatividade.

Conexão com o Futuro do Curso

Este ajuste mantém o sistema funcional enquanto o projeto continua evoluindo em direção a uma arquitetura mais robusta:

Estado Atual

O carrinho ainda está gerenciado dentro do componente `lista-produtos`. Isso funciona, mas limita o acesso ao estado em outros pontos da aplicação.

- Signal `carrinho` local ao componente
- Comunicação via `@Output()`
- Funcional para o estágio atual do curso

Próximos Passos

Em aulas futuras, o estado do carrinho será movido para uma arquitetura global, permitindo acesso de qualquer componente da aplicação.

- Estado movido para um **Service**
- Signal global compartilhado
- Arquitetura escalável e desacoplada

📄 Este ajuste é intencional e temporário. O objetivo é manter o sistema funcional a cada etapa do curso, evoluindo gradualmente sem deixar regressões acumuladas.

Conclusão

Este material mostrou um cenário real e muito comum no desenvolvimento de software profissional:



Regressão Corrigida

Identificamos e corrigimos uma regressão real introduzida durante refatorações do projeto de e-commerce.

5

Etapas de correção

Realizadas passo a passo



Comunicação Restaurada

Restauramos a comunicação entre componentes usando `@Output()` e event binding no template do pai.

3

Arquivos alterados

produto.ts, produto.html e lista-produtos.html



Conceitos Reforçados

Reforçamos conceitos fundamentais do Angular moderno: Signals, comunicação pai-filho e fluxo reativo.

2

Funcionalidades restauradas

Seleção e adição ao carrinho

Código final: produto.ts

```
import { Component, Input, Output, EventEmitter } from '@angular/core';
import { CurrencyPipe } from '@angular/common';

@Component({
  selector: 'app-produto',
  imports: [CurrencyPipe],
  templateUrl: './produto.html',
  styleUrls: ['./produto.css'],
})

export class Produto {

  @Input() nome: string = '';
  @Input() preco: number = 0;

  @Output() produtoSelecionado = new EventEmitter<string>();
  @Output() produtoAdicionado = new EventEmitter<{ nome: string; preco: number }>();
```

```
selecionarProduto() {
  this.produtoSelecionado.emit(this.nome);
}

adicionarAoCarrinho() {
  this.produtoAdicionado.emit({
    nome: this.nome,
    preco: this.preco
  });
}
```

Código final: produto.html

```
<div class="produto">
  <h3>{{ nome }}</h3>
  <p>{{ preco | currency: 'BRL' }}</p>

  <button (click)="selecionarProduto()">Selecionar {{ nome }}</button>
  <button (click)="adicionarAoCarrinho()">Adicionar ao carrinho</button>
</div>
```

Código final: lista-produtos.ts

```
import { Component, signal, computed, effect, inject } from '@angular/core';

import { ProdutosService } from '../produtos.service';

import { Produto } from '../produto/produto';

@Component({
  selector: 'app-lista-produtos',
  imports: [Produto],
  templateUrl: './lista-produtos.html',
  styleUrls: ['./lista-produtos.css'],
})
export class ListaProdutos {

  private produtosService = inject(ProdutosService);

  // SIGNALS

  produtos = signal<{ nome: string; preco: number }[]>([]);

  produtoSelecionado = signal<string | null>(null);

  carrinho = signal<{ nome: string; preco: number }[]>([]);

  carregando = signal(true);

  erro = signal<string | null>(null);
```

```
// COMPUTED

totalProdutos = computed(() => this.produtos().length);

valorTotal = computed(() => {

  return this.produtos().reduce((total, item) => total + item.preco, 0);

});

quantidadeCarrinho = computed(() => this.carrinho().length);

totalCarrinho = computed(() => {

  return this.carrinho().reduce((total, item) => total + item.preco, 0);

});

constructor() {

  this.carregarProdutos();

  effect(() => {

    console.log('Lista de produtos alterada:', this.produtos());

  });

  effect(() => {

    console.log('Valor total atualizado:', this.valorTotal());

  });

  effect(() => {

    if (typeof document !== 'undefined') {

      document.title = `(${this.totalProdutos()}) Minha Loja`;

    }

  });

}
```

Continuação - Código final: lista-produtos.ts

```
// MÉTODO HTTP (API)

carregarProdutos() {

  this.erro.set(null);

  this.carregando.set(true);

  this.produtosService.buscarProdutos().subscribe({

    next: (dados) => {

      const produtos = this.produtosService.transformarProdutos(dados);

      this.produtos.set(produtos);

      this.carregando.set(false);

    },

    error: (erro) => {

      console.error('Erro ao carregar produtos:', erro);

      this.erro.set('Erro ao carregar produtos. Verifique sua conexão e tente novamente.');
```

```
      this.carregando.set(false);

    },

  });
}
```

```
exibirProduto(nome: string) {

  this.produtoSelecionado.set(nome);

}

adicionarProduto() {

  this.produtos.update((listaAtual) => [...listaAtual, { nome: 'Teclado', preco: 250 }]);

}

substituirProdutos() {

  this.produtos.set([{ nome: 'Produto novo', preco: 999 }]);

}

adicionarAoCarrinho(produto: { nome: string; preco: number }) {

  this.carrinho.update((listaAtual) => [...listaAtual, produto]);

}

}
```

Código final: lista-produtos.html

```
<!-- Seção de Produtos -->

<section>

  <h2>Produtos:</h2>

  <!-- Estado 1: Loading -->

  @if (carregando()) {

    <p>Carregando produtos...</p>

  }

  <!-- Estado 2: Erro -->

  @if (erro()) {

    <p style="color: red">

      {{ erro() }}

    </p>

    <button (click)="carregarProdutos()">Tentar novamente</button>

  }

}
```

```
<!-- Estado 3: Sucesso -->

@if (!carregando() && !erro()) {

  <p>Total de produtos: {{ totalProdutos() }}</p>

  <hr />

  @for (item of produtos(); track $index) {

    <app-produto

      [nome]="item.nome"

      [preco]="item.preco"

      (produtoSelecionado)="exibirProduto($event)"

      (produtoAdicionado)="adicionarAoCarrinho($event)"

    />

  }

}

<h2>

  Produto selecionado:

  @if (produtoSelecionado()) {

    <strong style="color: green">

      {{ produtoSelecionado() }}

    </strong>

  } @else {

    <span style="color: gray">Nenhum</span>

  }

</h2>
```

```
<button (click)="adicionarProduto()">

  Adicionar Produto

</button>

<button (click)="substituirProdutos()">

  Substituir Lista

</button>

<h2>Total de produtos: {{ totalProdutos() }}</h2>

<h3>Valor total: R$ {{ valorTotal().toFixed(2) }}</h3>

</section>

<!-- Seção do Carrinho -->

<section>

  <hr />

  <h2>Carrinho:</h2>

  @for (item of carrinho(); track $index) {

    <p>{{ item.nome }} - R$ {{ item.preco.toFixed(2) }}</p>

  }

  <h3>Itens no carrinho: {{ quantidadeCarrinho() }}</h3>

  <h3>Total do Carrinho: R$ {{ totalCarrinho().toFixed(2) }}</h3>

</section>
```

Dúvidas

?

